

ArchSync: A Controlled Feasibility Study of Evidence-Backed Architecture Drift Detection in TypeScript Systems

Vo Duc Hieu*, Tran Minh Hoang[†], Ha Hoang Bach[‡], and Le Van Kiet[§]

*Software Engineering, FPT University, Ho Chi Minh City, Vietnam

Email: voduchieu@littleboys.biz; ORCID: 0009-0007-5389-5177

[†]Computer Science and Engineering, VNUK Institute for Research and Executive Education,
The University of Danang, Da Nang, Vietnam

Email: an1dee@littleboys.biz; ORCID: 0009-0000-0302-1841

[‡]Information Assurance, FPT University, Ho Chi Minh City, Vietnam

Email: bachcp6@littleboys.biz; ORCID: 0009-0000-5118-0660

[§]Software Engineering, VNUK Institute for Research and Executive Education,
The University of Danang, Da Nang, Vietnam

Email: levankiet1212.2004@littleboys.biz; ORCID: 0009-0007-8434-882X

Abstract—Software teams can keep builds green while implementation relationships drift away from an approved architecture, leaving reviewers without a repeatable way to decide whether a change is harmless, forbidden, or a legitimate evolution. This paper presents ArchSync, a deterministic architecture-conformance prototype that treats a machine-readable model as an executable contract and reconstructs an observed service graph from TypeScript source code. Stack-specific analyzers detect selected HTTP, PostgreSQL, Redis, and AMQP interactions, preserve file-level evidence, and feed a Git-diff gate that maps introduced findings to PASS, BLOCK, or REVIEW without rewriting the approved model. We verify the prototype on two co-developed development datasets and replay the same controlled changes through the incremental gate. Within these finite regression sets, the implementation agrees with the declared graph, decision, evidence-location, and full-scan oracles, and provenance tracking removes the targeted detector errors exposed by the earlier analyzer. An external-tool comparison remains proposed and was not executed within the reported evaluation; no accepted comparator result or common label mapping is available, and the study lacks an independently curated real-world holdout. Its results therefore establish executable-contract coherence and reproducible regression behavior only, not comparative advantage or general accuracy on TypeScript systems.

Index Terms—software architecture, architecture conformance, architecture drift, static analysis, architecture as code

I. INTRODUCTION

Software architecture defines intended components and permitted relationships. Evolution can introduce structural violations while compilation and functional tests remain green; mapping and practitioner studies describe erosion and continuing reliance on documentation and shared practices [1], [2]. A seemingly local change can bypass a gateway, access a database directly, or add infrastructure without an architecture decision.

ArchSync connects a versioned expected graph from `architecture.yaml` to an observed graph reconstructed

from source. Explicit deny, allow, require, and require-path rules distinguish broken constraints from non-forbidden topology changes. Evidence links findings to implementation and model locations. For pull requests, only findings introduced relative to the Git merge base determine PASS, BLOCK, or REVIEW; historical findings remain visible. The observed graph follows code, while the expected graph remains approved intent. REVIEW requests a human decision and does not itself implement approval or rejection governance.

The prototype detects selected TypeScript/Node.js HTTP, PostgreSQL, Redis, and AMQP patterns. The model and conformance contracts are language-neutral, but only this source analyzer has empirical evidence. Determinism makes repeated explicit-rule decisions reproducible; it does not imply complete extraction or general accuracy. AI explanations, automatic repairs, IaC, and runtime observations remain outside the evaluated decision path.

The contributions are: (1) a typed executable-contract prototype with source/model evidence, (2) a relation-level incremental PR gate, and (3) a controlled verification artifact with explicit units and provenance. They are bounded prototype contributions, not new conformance concepts or validated industrial effectiveness.

II. BACKGROUND AND RELATED WORK

This scoped narrative synthesis establishes context for ArchSync’s design; it is not a systematic literature review. Sources were selected purposively from the retained keyword-search corpus for architecture conformance, drift detection, and static-analysis reliability, emphasizing 2022–2025. Four earlier works supply concept origins. Selection and interpretation may omit relevant approaches; coverage completeness and absence-of-prior-work claims are outside scope.

Erosion and reconstruction. A mapping of architecture erosion describes its manifestations and consequences [1]. Code-review studies identify symptoms in OpenStack and later OpenStack/Qt projects [3], [4]; their shared project contexts are not independent replications. Practitioner evidence identifies documentation, responsibilities, and shared practices as ways to manage drift [2]. Reconstruction taxonomies organize recovery strategies [5], [6], while a microservice mapping compares tool purposes and availability [7]. View-based analysis compares intended and reconstructed specifications in an industrial case [8]. ArchSync restricts its observation boundary to explicit service and infrastructure relationships.

Conformance. Reflexion Models, static conformance comparisons, and DCL provide the origins of model comparison and allowed/forbidden dependency rules [9]–[11]; neither concept is new here. CALint applies dependency-rule checks to Python [12]. ArchLintor is a close TypeScript comparison, detecting module-interaction violations and supporting CI/CD [13]. ArchSync’s typed service topology differs from that module boundary; no head-to-head accuracy result is claimed. IaC metrics address deployment coupling [14], and 3Erefactor searches for executable inconsistency-reducing repairs [15]. LLM-generated architectural tests distinguish execution success from correct rule expression [16]. ArchSync executes explicit rules deterministically, but that choice alone implies neither accuracy nor superiority.

Change-time feedback. Commit-level prediction studies architectural change [17]; repository mining tracks microservice evolution [18]; industrial conformance visualization supplies practitioner evidence [19]. A predicted or observed change need not violate intended architecture. ArchSync instead reports findings introduced at a Git head, preserving separate violation and evolution decisions.

Evaluation reliability. Replication support remains an evaluation concern [20]. Decomposition surveys cover an adjacent problem [21], whereas a microservice-recovery comparison executes nine tools and reports individual and combined recovery performance [22]. That study supplies an external-comparison precedent, not scores directly comparable with ArchSync. Static visual models and multiple dependency models motivate future multi-source work [23], [24]. Interrogation testing exposes both precision and soundness failures [25], motivating our explicit hard negatives. These sources establish comparison dimensions rather than exhaustive novelty or representative accuracy.

III. PROBLEM DEFINITION

Let $M = (C_E, E_E, R)$ contain expected components, typed directed edges, and rules. An edge is (u, t, v) : source, relation type, and target. Given repository S and supplied model M , the analyzer constructs $A(S; M) = (C_O, E_O, P)$, where P links observations to file, line, column, detector, excerpt, and a fixed confidence annotation (not a calibrated probability).

Deny rejects matching edges; *allow* restricts selected sources to an allowed target set; *require* demands a direct

matching edge for every selected source; *require-path* demands a non-empty directed path. Selectors may use wildcards and constrain relation type. Given graph difference Δ and violation set $\mathcal{V}$, classification is

$$\text{class}(M, A(S; M)) = \begin{cases} \text{violation,} & |\mathcal{V}| > 0, \\ \text{evolution,} & |\mathcal{V}| = 0 \wedge |\Delta| > 0, \\ \text{no-impact,} & \text{otherwise.} \end{cases}$$

For Git base S_b and head S_h , findings are compared by stable semantic identity:

$$\mathcal{F}_b = \mathcal{F}(M, A(S_b; M)), \quad \mathcal{F}_h = \mathcal{F}(M, A(S_h; M)),$$

$$K_b = \{k(f) : f \in \mathcal{F}_b\}, \quad \mathcal{F}^+ = \{f \in \mathcal{F}_h : k(f) \notin K_b\}.$$

The analyzer maps paths and metadata using the model; this is not autonomous boundary discovery. The key $k(f)$ contains kind, rule identifier, change, component, and edge key; coordinates are excluded. Another call site for an old edge need not introduce a finding. Only introduced findings control the gate: a rule finding yields BLOCK, otherwise an evolution finding yields REVIEW, otherwise PASS. Common findings remain visible as pre-existing; base-only findings are resolved. PASS does not establish a violation-free head or complete observation.

IV. RESEARCH QUESTIONS

The feasibility study asks: **RQ 1**, how many expected components, relationships, and detector signals are reconstructed; **RQ 2**, whether no-impact/violation/evolution labels and violated rule sets match; **RQ 3**, whether call-site and missing-relation anchor files/lines match separately at the case level; and **RQ 4**, whether normalized outputs replay identically and incremental/full-scan results agree, with what parsed scope and recorded cold/warm latency. D1 uses matched/extra/missed counts; D2 additionally uses descriptive signal-level precision, recall, F 1, and specificity. All questions are bounded by the development corpora. This post-hoc reporting clarification does not alter frozen inputs, labels, or acceptance rules.

V. PROPOSED APPROACH

Executable intent and evidence. A versioned YAML model defines component identifiers, layers, typed relationships, rules, severity, and ownership. Schema and semantic checks reject malformed models, unknown references, duplicate edges, and invalid identifiers. Expected edges are canonicalized; diagrams are derived views, not separate authorities. Findings retain a contract version, stable identifier, kind, severity, optional rule, affected element, source evidence, and model evidence. Forbidden-edge evidence identifies a detected call. Missing-edge/path evidence uses a deterministic component anchor because an absent call has no source location.

Observation. Guardian parses TypeScript under declared component roots with the Compiler API. Detectors inspect imports, variable initializers, calls, and endpoint expressions for selected HTTP, PostgreSQL, Redis, and AMQP patterns. PostgreSQL/Redis operations require a receiver associated

with a recognized client; AMQP requires a channel from a recognized connection. These per-file name associations filter unassociated method lookalikes but do not establish soundness under arbitrary reassignment or shadowing. Endpoints use literals and fixed fallbacks, not runtime execution. Targets use hostname only, ignoring port/path. Unresolved HTTP produces no edge and no explicit unknown finding. AMQP paths are broker-level, not proof of matching queues or message delivery. A typed require-path requires that type on every edge. Observations and evidence are sorted and deduplicated.

Fixed precedence. Any rule violation outranks evolution, regardless of severity; both finding types remain inspectable. This conservative policy prevents a forbidden edge from being treated only as a reviewable addition, but can delay acceptable changes bundled with it. Configurable severity/context policies are alternatives, not evaluated features. The command returns 0/1/3 for PASS/BLOCK/REVIEW. A repository must separately enforce those outcomes.

Incremental gate and review lifecycle. Guardian resolves a merge base, maps changed paths to source components, and caches the baseline graph by base commit, model hash, and analyzer version. It reparses all TypeScript files in affected components, replacing their observations while retaining unaffected ones. Component-wide scanning preserves independently detectable relationships; it does not add inter-file data-flow analysis. The same supplied model is used to analyze and evaluate base and head; a model edit invalidates the cache, not a separate model-to-model delta. Cache checks assume trusted local graph contents. Source annotations and a report support inspection. In existing D1 case 06, a frontend fetch at `app.ts:14` creates a payment-service HTTP edge forbidden by ARCH-001; its new key yields BLOCK. This illustration adds no evaluation sample.

The analyzer never rewrites the expected model. Accepting evolution requires a separately reviewed model change. The evaluated gate neither authenticates approval nor stores rejection: unchanged code/model still returns REVIEW after a human rejects the proposal. Repository controls must prevent the merge; the developer revises or withdraws the code and reruns. Persisting the rationale or adding a lasting constraint is a proposed governance action, not implemented automatic rejection handling.

VI. SYSTEM ARCHITECTURE

Figure 1 separates approved intent from base and proposed implementation state. ArchSync Core validates the model and provides the expected graph and executable rules. Guardian caches a base Observed Graph, scopes the Git diff to affected source components, incrementally constructs the head graph, and preserves source evidence. The gate compares base and head findings before deriving the pull-request decision.

The architecture is divided across focused repositories. `archsnc-core` contains the architecture contract and deterministic conformance functions. `archsnc-guardian` contains analyzers and findings. `archsnc-benchmark` contains the controlled development benchmark. `archsnc-examples` contains

readable models and generated views. `archsnc-mcp` is reserved for a later interaction layer and is excluded from the initial experiment. The named draft exposes these real repository links; the double-blind build replaces them with blinded artifact labels.

VII. IMPLEMENTATION

A. Prototype and Detector Logic

`archsnc-core` implements the architecture contract, schema and semantic validation, normalized graph comparison, and deny/allow/require/require-path rules. `archsnc-guardian` implements TypeScript extraction, source evidence, and the Git-diff gate. Guardian delegates conformance semantics to Core so repository and incremental checks use the same rule engine.

For a concrete PostgreSQL example, consider the following code inside the `order-service` component:

```
import { Pool as PgPool } from "pg";
const db = new PgPool({
  connectionString:
    "postgres://postgres:5432/orders"
});
await db.query("select 1");
```

The import detector records `PgPool` as a local name for a recognized `pg` constructor. The initializer pass recognizes `new PgPool`, resolves the inline `connectionString`, and associates `db` with target `postgres`, derived from the URL hostname. At `db.query`, the call detector requires that receiver association before emitting the directed edge (`order-service,data,postgres`). Merely importing `pg`, constructing an unrelated client, or calling an unrelated object's `query` method does not satisfy this detector. Construction alone does not emit the database-access edge. Evidence identifies the query call's file, line, column, detector, and excerpt.

The analysis uses per-file identifier maps and four passes over variable declarations; it is not general alias analysis or interprocedural data-flow analysis. Imported client wrappers, arbitrary reassignment, shadowed identifiers, and relationships requiring resolution across files are not comprehensively handled. Endpoint resolution uses literals, mapped variables, and naming conventions such as `DATABASE_URL` for `postgres`. For binary expressions it tries the right-hand expression first; this deterministic heuristic does not evaluate runtime environment values or branch conditions. The PostgreSQL evidence confidence of 0.99 is a fixed detector annotation, not a calibrated probability. Redis operations similarly require a recognized client, and AMQP operations require a channel associated with a recognized connection.

B. Controlled Evaluation Artifact

`archsnc-benchmark` supplies the synthetic Order Platform, 20 executable patches with group-authored ground truth, and the 40-signal positive/hard-negative corpus. Each patch runs on a fresh copy. Git-diff experiments additionally construct a baseline commit, perform cold and warm checks, and

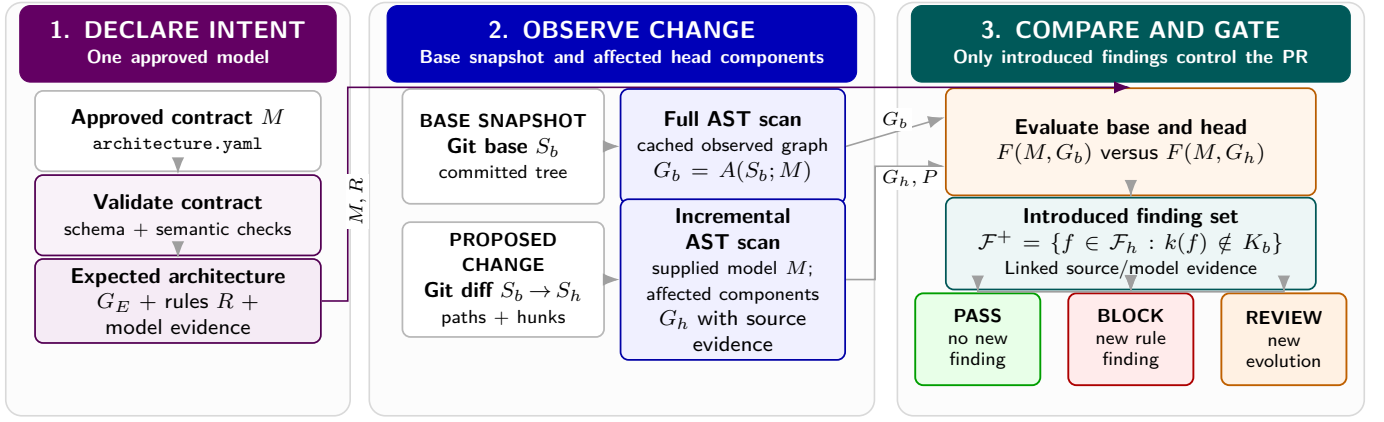


Fig. 1. ArchSync Phase 3 system architecture. The three-stage pipeline separates approved intent, base/head observation, and the pull-request gate. The model remains authoritative; only findings introduced by the Git diff determine PASS, BLOCK, or REVIEW.

compare incremental head observations with a separate full scan. This full scan checks incremental equivalence; it does not independently establish extraction accuracy.

Measurements use Core commit 2affbbb0da85, Guardian commit 8779bf7965b3, and Benchmark commit 24d63ebf2fc3. Later development heads do not replace these measured artifacts. Source/patch manifests, raw results, package hashes, historical verification commands, and the detector/gate audit remain in the supplementary artifact. These controls support inspection, not independent accuracy evidence. `archsnc-mcp` is excluded.

VIII. CONTROLLED VERIFICATION METHODOLOGY

D1 development benchmark. An Order Platform baseline has five components and five directed relationships. Twenty isolated patches comprise nine no-impact changes, seven violations, and four evolutions. Cases include service bypass, required-edge removal, allow and require-path rules, PostgreSQL aliases, Redis and AMQP additions, a new service, and non-architectural lookalikes. Each declares changed files, graph delta, classification, rule identifiers, and source evidence. Topology intent originated before analyzer integration, but patches and exact evidence lines were refined alongside it. D1 is a co-developed regression oracle, not a blinded holdout.

D2 detector corpus. Five detector groups (HTTP, PostgreSQL, Redis, AMQP publish, and AMQP consume) each contain four positive and four hard-negative source lines: 40 annotated signals in total. Positives exercise aliases, namespaces, environment fallbacks, and supported operations. Negatives include unused imports, custom functions, method-name collisions, comments, and string literals. An annotation binds detector, file, line, expected edge, and source substring. D2 was authored from known v 0.1 weaknesses and guided v 0.2, so the paired result measures targeted regression behavior.

Execution and P3 replay. D1 validates model, manifest, hashes, and patch applicability, then executes an unchanged baseline twice and each individually patched copy twice: 42 analyzer runs. D2 executes each of two pinned analyzer

versions twice. P3 creates a fresh Git baseline for every D1 patch, applies it, and runs one cold and one warm diff check. A separate full-head scan provides an equivalence oracle. The same 20 patches are reused, not counted as a third independent dataset. Normalized duplicate outputs exclude timing and cache state. The functional workflow was reproduced on clean Linux and Windows CI runners; latency comes only from the recorded Windows environment.

Measures. D1 reports matched, extra, and missed graph items, separating changed items from repeated baseline structure. Empty changed sets contribute no positive item. Classification uses all 20 cases; rule agreement uses seven violations; evidence agreement splits the original 11 cases post hoc into nine call-site and two missing-relation anchor cases (08 and 17). It requires declared file/line matches per case, not per-finding recall; labels and acceptance rules are unchanged. D2 counts annotated detector signals, not unique edges, with

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN},$$

$$F_1 = \frac{2PR}{P + R}, \quad Sp = \frac{TN}{TN + FP}.$$

Specificity covers only declared hard negatives. P3 checks decision, changed-file and architecture-delta equality, full-scan equivalence, replay, and expected cache miss/hit. Parsed scope is the ratio of affected-component TypeScript file instances to full-head instances. Latency uses nearest-rank $x_{(\lceil pn \rceil)}$ at p50/p95. With 20 samples, p50 is the tenth observation, not the average of the middle pair. Original timings are descriptive, not population or comparative-performance estimates. Node 22.16.0, Windows 10.0.26200, i7-13650HX and 20 logical CPUs were recorded. RAM, power mode, background/thermal load and filesystem-cache state were not recorded.

Engineering gates. The existing protocol required D1 precision/recall at least 0.85, exact decisions/rules/evidence, and deterministic replay; D2 required all annotated positives and negatives correct; P3 required exact replay/full-scan agreement and cache behavior. These are development acceptance rules,

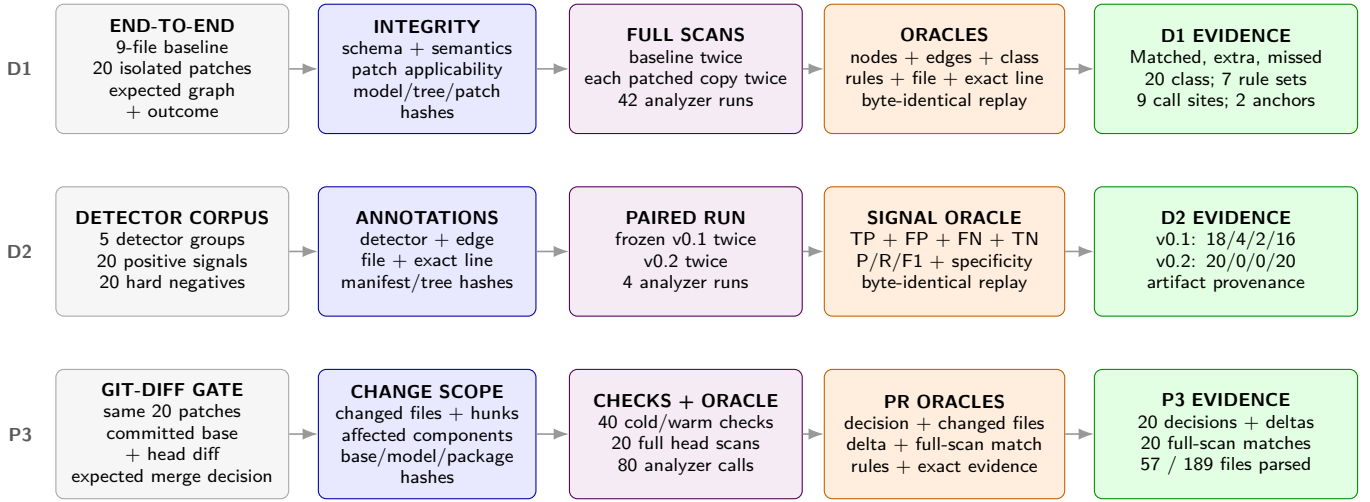


Fig. 2. Controlled verification: full-graph D1 changes, D2 source signals, and incremental P3 replay. All routes use co-developed data.

TABLE I

D1 COUNTS ON 20 CO-DEVELOPED PATCHES. THE FIVE CHANGED NODES AND 12 CHANGED EDGES ARE REGRESSION ITEMS, NOT INDEPENDENT SAMPLES OF GENERAL ACCURACY; FULL-GRAPH COUNTS REPEAT BASELINE STRUCTURE.

Measure	Matched	Extra	Missed
Full-graph nodes	105	0	0
Full-graph edges	108	0	0
Changed nodes	5	0	0
Changed edges	12	0	0

TABLE II

DECISION, EVIDENCE, AND REPLAY OUTCOMES ON CONTROLLED D1 CASES.

Measure	Matches	Total
Classification	20	20
Violation rule set	7	7
Call-site file and line	9	9
Missing-relation anchor file and line	2	2
Deterministic cases	20	20

not statistical standards. The paper reports counts even where its underlying engineering gate computes rates.

Proposed external comparison. External comparison remains future work; no external-tool comparison was executed within the reported evaluation; no accepted comparator result or common label mapping is available. A module-dependency analyzer is a candidate, but module imports and typed service calls need an explicit, independently reviewed mapping before comparative scoring. The proposed protocol must freeze repositories, versions, configurations, dependency-resolution conditions, and independently authored ground truth before inspecting outputs. Unsupported relations must be separated from misses within common supported scope. The protocol rejects a comparator without a non-empty semantic intersection and requires separately freezing any adapter. A capability discussion cannot complete the baseline or supply an accuracy estimate. Reusing D1 would remain a development-set comparison rather than an independent holdout.

IX. CONTROLLED VERIFICATION RESULTS

RQ1: graph reconstruction and detector signals. The unchanged repository was reconstructed as five components and five relationships with a no-impact classification. Table I reports no false positive or false negative in D1. Full-graph totals aggregate 105 expected nodes and 108 expected edges across 20 patched repositories. Only five node changes and 12

TABLE III

P3 GIT-DIFF GATE OUTCOMES AND MEASURED SCOPE ON THE D1 REPLAY.

Measure	Result
Classification agreement	20/20
Merge-decision agreement	20/20
Changed-file agreement	20/20
Architecture-delta agreement	20/20
Incremental/full-scan agreement	20/20
Violation rule-set agreement	7/7
Call-site exact line	9/9
Anchor exact line	2/2
Cache hit on repeated check	20/20
Normalized cold/warm replay	20/20
Incremental files / head files	57/189
Cold p50 / p95 (ms)	518.51 / 531.05
Warm p50 / p95 (ms)	242.62 / 249.30

edge changes contribute positive changed-graph items. These finite, co-developed cases test contract agreement; they do not estimate performance on unseen projects.

Table IV gives the signal-level regression result. On D2, v0.1 achieved precision 0.818, recall 0.900, F1 0.857, and specificity 0.800. Its four false positives were a PostgreSQL-shaped local query, two AMQP-publish lookalikes, and an AMQP-consume lookalike; its two false negatives were Redis hSet and lPush. On the same annotated signals, v0.2 matched all 20 positives and rejected all 20 annotated negatives (precision, recall, F1, and specificity each 1.000 on this cor-

TABLE IV
D2 WITHIN-TOOL REGRESSION CONFUSION MATRICES FOR THE FROZEN v0.1 AND PROVENANCE-AWARE v0.2 ANALYZERS ON THE SAME CO-DEVELOPED 40-SIGNAL CORPUS.

Detector	v0.1				v0.2			
	TP	FP	FN	TN	TP	FP	FN	TN
fetch	4	0	0	4	4	0	0	4
PostgreSQL	4	1	0	3	4	0	0	4
Redis	2	0	2	4	4	0	0	4
AMQP publish	4	2	0	2	4	0	0	4
AMQP consume	4	1	0	3	4	0	0	4
Overall	18	4	2	16	20	0	0	20

pus). Package-provenance tracking removed the four method-name false positives, while the expanded Redis operation set recovered the two missed positives. Because D2 was constructed from known v0.1 weaknesses, this is targeted regression evidence and not an estimate over arbitrary TypeScript syntax or an external baseline comparison.

RQ2: change classification. ArchSync matched all 20 D1 labels: nine no-impact, seven violation, and four evolution cases. It also matched the complete expected rule-identifier set for all seven violations. The added allow and require-path cases exercise rule semantics beyond direct deny and require checks. Redis, AMQP, Fraud Service, and payment-cache additions were classified as evolution rather than violation, preserving a human-review path.

RQ3: evidence localization. All 11 finding-bearing D1 cases are split post hoc into call-site file/line agreement in 9/9 cases and anchor agreement in 2/2 cases, for missing-edge case 08 and missing-path case 17. Both P2 and P3 satisfy the separate checks. These are case-level outcomes: case 15 has two violated rules but contributes one case. An anchor is not a localized absent call, and neither measure evaluates ranking, all supporting locations, or reviewer effort.

RQ4: determinism and reproducibility. The D1 baseline and all 20 cases produced byte-identical normalized JSON across duplicate executions, giving 42 current-version executions with no replay mismatch. Both D2 analyzer versions were also deterministic across their duplicate runs. P3 produced identical normalized cold and warm results for 20/20 cases, matched all 20 declared architecture deltas, matched a separate full head scan in all 20 cases, and hit the baseline cache on every repeated check. As Table III shows, component-incremental analysis parsed 57 of 189 TypeScript file instances, $r_{\text{parse}} = 0.3016$. On the recorded Windows environment, warm nearest-rank p50 latency was 242.62 ms compared with 518.51 ms cold, a descriptive 53.2% reduction; warm p95 was 249.30 ms compared with 531.05 ms cold. Local verification reproduced all stored results and repository gates. GitHub Actions independently reproduced the functional benchmark, detector, and Git-diff evidence on both ubuntu-latest and windows-latest; timing values were not treated as cross-platform acceptance thresholds.

Boundary of the reported evidence. The observations above come from D1, D2, and the P3 reuse of D1. No external-tool result is accepted into this evaluation, and no common la-

bel mapping has been established for comparative scoring. The internal v0.1–v0.2 detector comparison and incremental/full-scan equivalence therefore answer different questions from an external baseline: the former exposes targeted regression behavior, while the latter checks whether scoped execution preserves this implementation’s full-scan result.

Exact agreement with these oracles does not establish that the expected architecture is complete or appropriate for another system. In particular, a missed relationship outside the detector vocabulary can be absent from both incremental and full scans without producing an equivalence mismatch. Similarly, duplicate executions can reproduce the same incorrect inference. The graph, evidence-location, decision, and replay checks are complementary consistency checks; they do not substitute for independently authored semantic labels or a separately evaluated tool.

X. DISCUSSION

The results establish agreement between implemented contracts and their development oracles. D2 shows why end-to-end graph counts alone are insufficient: repeated baseline structure can dominate totals, while hard-negative source locations expose method-name collisions. Its six corrected v 0.1 errors support targeted diagnosis and regression protection, not an unbiased estimate of improvement on unseen programs.

The three-way decision separates forbidden relationships from changes requiring a design decision. P3 reports historical findings while gating only introduced findings, permitting gradual adoption without treating historical keys as new findings. REVIEW neither approves evolution nor rewrites the expected model. The evaluated implementation has no durable rejection disposition: rejecting an evolution requires changing or withdrawing the proposed code and rerunning the check; unchanged inputs still yield REVIEW. Accepting it requires a separately reviewed model change. Owner protection is therefore an organizational prerequisite, not an experimentally validated property of the gate.

Component scoping parsed 57/189 file instances and warm caching reduced the recorded nearest-rank p50 from 518.51 to 242.62ms. Those observations combine parsing, Git processes, cache I/O, comparison, and reporting; they do not isolate algorithmic cost or establish scale. Next experiments require independently annotated real repositories and histories, capability-aligned external baselines, and a practitioner

pilot measuring false-block burden, review effort, and model-change governance. IaC, runtime, AI assistance, and other language analyzers remain unevaluated extensions.

XI. THREATS TO VALIDITY

Construct validity. Typed topology and explicit rules do not measure responsibility, rationale, runtime quality, or developer usefulness. Evidence agreement requires one matching location, not complete or well-ranked evidence. An absent relationship has only a deterministic component anchor. D2 signals can share an edge, and its negatives do not represent all unsupported syntax. Parsed-file counts measure analysis scope, not CPU work.

Internal validity. The authors developed the analyzer and both datasets. D1 topology intent preceded integration, but patch implementations and exact evidence lines co-evolved with the analyzer. D2 targeted known v0.1 errors and guided v0.2. Replaying D1 in P3 adds incremental checks, not independent observations. Hashes, raw predictions, mutation checks, and duplicate execution expose inconsistency but do not establish independent ground truth.

External and conclusion validity. One small synthetic system, 20 patches, and 40 curated signals cannot estimate general TypeScript accuracy. The five changed nodes, 12 changed edges, seven violation rule sets, and 11 evidence cases are finite regression items. Wrappers, dynamic endpoints, dependency injection, generated code, unconventional repositories, other languages, and runtime-only relationships remain outside evaluated coverage. Latency is descriptive from one environment; there is no workload-population inference, scalability study, or significance test. No industrial deployment or developer study was executed. Later repository protection is not experimental evidence of a non-bypassable human-governed workflow.

Comparative validity. The external comparison remains proposed and contributes no reported execution or result. No accepted common label mapping exists. Differences between import and service-call semantics require a reviewed scoring abstraction. Independently authored data and capability-matched ground truth remain necessary. Duplicate replay can reproduce an incorrect inference, and incremental/full-scan agreement can preserve omissions shared by both modes. These consistency checks cannot establish semantic completeness.

Literature positioning. The narrative review carries source-selection and interpretation bias and may omit approaches. Coverage completeness and absence-of-evidence claims are outside its design, not deferred SLR deliverables. Related studies can share projects or implementations, so citation breadth does not imply independent replication.

Reliability. Pinned artifacts, normalized ordering, isolated patches, duplicate execution, and clean Linux/Windows functional checks support reproducibility. The frozen v0.1 result is provenance-validated rather than continuously regenerated; Windows timing is not a cross-platform performance claim.

These controls address repeatability of the controlled experiment, not the independence of its labels.

XII. CONCLUSION AND FUTURE WORK

ArchSync implements a deterministic bridge between a declared architecture contract and a source-derived TypeScript service graph. On the co-developed development datasets, its graph, classification, evidence-location, replay, and incremental/full-scan outputs agree with the declared regression oracles. The targeted detector corpus also exposes concrete errors in the earlier analyzer and verifies that provenance tracking addresses those cases.

They do not establish general accuracy, comparative advantage, or governance effectiveness: the datasets co-evolved with the implementation, no accepted external comparison or common label mapping is available, and no independent real-world holdout exists. The analyzer never rewrites the model; evolution needs human review. Next studies require an independent multi-repository holdout, capability-matched external comparison and real pull-request pilot. Only after those gates produce auditable evidence should claims extend to external accuracy, performance or developer value.

AUTHOR INFORMATION AND CONTRIBUTIONS

Vo Duc Hieu (ORCID: 0009-0007-5389-5177) - Conceptualization; Methodology; Software; Project administration; Writing.

Tran Minh Hoang (ORCID: 0009-0000-0302-1841) - Software; AI methodology; Validation.

Ha Hoang Bach (ORCID: 0009-0000-5118-0660) - Investigation; Data curation; Formal analysis; Evaluation.

Le Van Kiet (ORCID: 0009-0007-8434-882X) - Software; Validation; Reproducibility.

REFERENCES

- [1] R. Li, P. Liang, M. Soliman, and P. Avgeriou, "Understanding software architecture erosion: A systematic mapping study," *Journal of Software: Evolution and Process*, vol. 34, no. 3, p. e2423, 2022.
- [2] E. Anthony, A. Berntsson, T. Santilli, and R. Wohlrab, "We're drifting apart: Architectural drift from the developers' perspective," in *2024 IEEE 21st International Conference on Software Architecture*. Hyderabad, India: IEEE, 2024, pp. 101–111.
- [3] R. Li, M. Soliman, P. Liang, and P. Avgeriou, "Symptoms of Architecture Erosion in Code Reviews: A Study of Two OpenStack Projects," in *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. IEEE, 2022, pp. 24–35.
- [4] R. Li, P. Liang, and P. Avgeriou, "Warnings: Violation symptoms indicating architecture erosion," *Information and Software Technology*, vol. 164, p. 107319, 2023.
- [5] S. Ducasse and D. Pollet, "Software architecture reconstruction: A process-oriented taxonomy," *IEEE Transactions on Software Engineering*, vol. 35, no. 4, pp. 573–591, 2009.
- [6] A. Qayum, M. Zhang, S. Colreavy, M. Chochlov, J. Buckley, D. Lin, and A. R. Sai, "A Framework and Taxonomy for Characterizing the Applicability of Software Architecture Recovery Approaches: A Tertiary-Mapping Study," *Software: Practice and Experience*, 2024, first published online 17 July 2024.
- [7] A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, and D. Taibi, *Tools Reconstructing Microservice Architecture: A Systematic Mapping Study*. Springer Nature Switzerland, 2024, pp. 3–18. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-66326-0_1
- [8] B. Uzun and B. Tekinerdogan, "Detecting deviations in the code using architecture view-based drift analysis," *Computer Standards & Interfaces*, vol. 87, p. 103774, 2024.

- [9] G. C. Murphy, D. Notkin, and K. J. Sullivan, "Software reflexion models: Bridging the gap between source and high-level models," in *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering*. New York, NY, USA: ACM, 1995, pp. 18–28.
- [10] J. Knodel and D. Popescu, "A comparison of static architecture compliance checking approaches," in *Sixth Working IEEE/IFIP Conference on Software Architecture*. Los Alamitos, CA, USA: IEEE, 2007, pp. 12–21.
- [11] R. Terra and M. T. Valente, "A dependency constraint language to manage object-oriented software architectures," *Software: Practice and Experience*, vol. 39, no. 12, pp. 1073–1094, 2009.
- [12] C. O. de Oliveira Junior, J. Carvalho, F. F. Silveira, T. S. da Silva, and E. M. Guerra, "CALint: A Tool for Enforcing the Clean Architecture's Dependency Rule in Python," in *Computational Science and Its Applications – ICCSA 2022 Workshops: Malaga, Spain, July 4–7, 2022, Proceedings, Part V*. Springer International Publishing, 2022, pp. 534–549.
- [13] M. Resende, R. Durelli, and R. Terra, "ArchLintor: detecting architectural violations in TypeScript systems," in *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*, ser. SBES 2025. Sociedade Brasileira de Computação, Sep. 2025, pp. 900–906. [Online]. Available: <http://dx.doi.org/10.5753/sbes.2025.11075>
- [14] E. Ntontos, U. Zdun, J. Soldani, and A. Brogi, *Assessing Architecture Conformance to Coupling-Related Infrastructure-as-Code Best Practices: Metrics and Case Studies*. Springer International Publishing, 2022, pp. 101–116. [Online]. Available: http://dx.doi.org/10.1007/978-3-031-16697-6_7
- [15] J. Liu, W. Jin, J. Zhou, Q. Feng, M. Fan, H. Wang, and T. Liu, "3Erefactor: Effective, Efficient and Executable Refactoring Recommendation for Software Architectural Consistency," *IEEE Transactions on Software Engineering*, vol. 50, no. 10, pp. 2633–2655, Oct. 2024. [Online]. Available: <http://dx.doi.org/10.1109/tse.2024.3449564>
- [16] A. Lucena, E. L. G. Alves, and J. Brunet, "Language Models as Architectural Gatekeepers: Automating Conformance Checking from Natural Language," in *Anais do XXXIX Simpósio Brasileiro de Engenharia de Software (SBES 2025)*. Sociedade Brasileira de Computação, 2025, pp. 713–719.
- [17] W. Zhan, R. Mo, Y. Jiang, and D. Wang, "Just-in-Time Prediction of Software Architectural Changes Through Commit-Level Analyses," *IEEE Transactions on Software Engineering*, vol. 51, no. 8, pp. 2285–2304, Aug. 2025. [Online]. Available: <http://dx.doi.org/10.1109/tse.2025.3587849>
- [18] P. Genfer and U. Zdun, "Exploring Architectural Evolution in Microservice Systems Using Repository Mining Techniques and Static Code Analysis," in *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, September 3–6, 2024, Proceedings*. Springer Nature Switzerland, 2024, pp. 157–173.
- [19] F. Zamfirov, A. Radulescu, J. Krüger, and M. R. V. Chaudron, "Lessons from Visualizing Software Architecture Structure Conformance at Thermo Fisher Scientific," in *Software Engineering and Advanced Applications: 51st Euromicro Conference, SEAA 2025, Salerno, Italy, September 10–12, 2025, Proceedings, Part III*. Springer Nature Switzerland, 2025, pp. 372–389.
- [20] M. Konersmann, A. Kaplan, T. Kühn, R. Heinrich, A. Koziol, R. H. Reussner, J. Jürjens, M. al Doori, N. Boltz, M. Ehl, D. Fuchß, K. Großer, S. Hahner, J. Keim, M. Lohr, T. Sağlam, S. Schulz, and J.-P. Töberg, "Evaluation methods and replicability of software architecture research objects," in *2022 IEEE 19th International Conference on Software Architecture (ICSA)*. Honolulu, HI, USA: IEEE, 2022, pp. 157–168.
- [21] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, "Decomposition of monolith applications into microservices architectures: A systematic review," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, 2023.
- [22] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, and D. Taibi, "Comparison of static analysis architecture recovery tools for microservice applications," *Empirical Software Engineering*, vol. 30, no. 5, Jun. 2025. [Online]. Available: <http://dx.doi.org/10.1007/s10664-025-10686-2>
- [23] T. Cerny, A. S. Abdelfattah, J. Yero, and D. Taibi, "From static code analysis to visual models of microservice architecture," *Cluster Computing*, vol. 27, no. 4, pp. 4145–4170, Apr. 2024. [Online]. Available: <http://dx.doi.org/10.1007/s10586-024-04394-7>
- [24] M. Altınışık, H. Sözer, and G. Gürsun, "Software Architecture Recovery from Multiple Dependency Models," in *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. ACM, 2024, pp. 1185–1192.
- [25] D. Kaundlstorfer, A. Isychev, V. Wüstholtz, and M. Christakis, "Interrogation testing of program analyzers for soundness and precision issues," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. New York, NY, USA: ACM, 2024, pp. 319–330.